Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

ADE

Agile Development Environment (ADE) is modular Docker-based tool to ensure that all developers in a project have a common, consistent development environment.

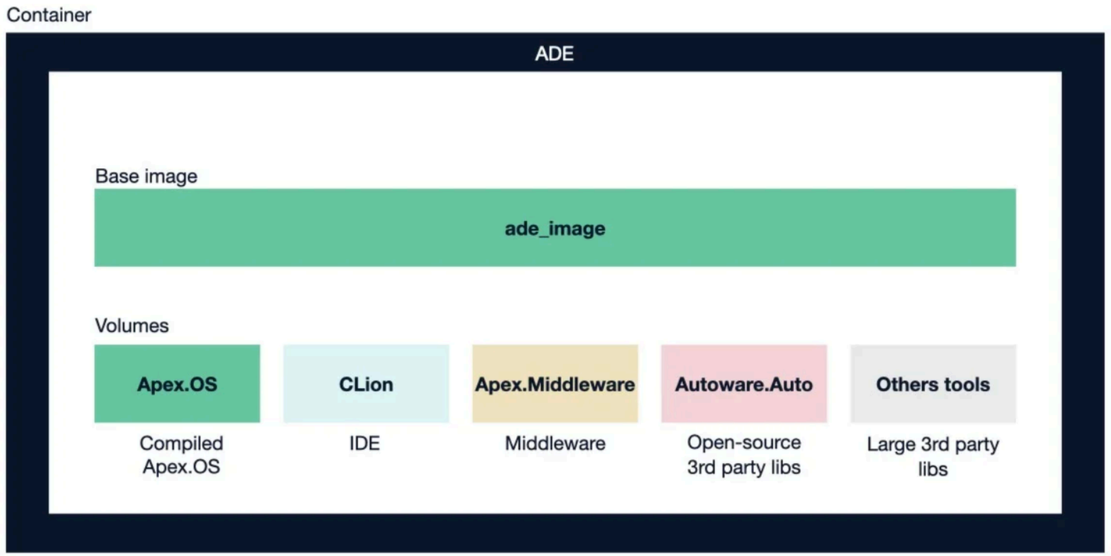
In other words, ADE leverages Docker containers as a way to create a reproducible development environment, simplifying the process of getting new developers up and running.

ADE works in such a way that developers do not need to have any knowledge about Docker to get started: **ade start** starts the container, **ade enter** enters the container, and **ade stop** stops the container; anything that should persist between restarts (e.g. a repository clone, log files, etc.) is kept in the home directory, which maps to a directory on the host

ADE

Once inside the container, developers have the correct version of development tools and third-party dependencies installed and ready to use, and they can launch editors and IDEs such as Atom, VSCode, and CLion.

If the developer wants to try out a tool (or update to a newer version of a library), they can install it, knowing that rolling back will simply require a restart of ADE (**ade stop** and **ade start**, or **ade start -f**). Alternatively, if the new tool proves to be useful, making it available to the rest of the team is as simple as updating the Docker image, an update which other developers can then download with **ade start --update**.



ADE CLI

ADE integrates with the Continuous Integration (CI) pipeline.

- ade-cli, the ADE image, and ADE.
  - **ade-cli** is a command line tool that provides the commands **ade start**, **ade enter**, and **ade stop**.
  - **ADE image** is the Docker image which **ade-cli** uses to create a container; this image needs to be built in an ADE-compatible way
  - **ADE** refers to a container started from the ADE image with **ade-cli**.

ADE e Docker

ADE does not hide away Docker; ADE allows anyone with knowledge of Docker to leverage features that are not provided by **ade-cli** out of the box.

For example, when working with real sensors, developers need to mount USB devices, open ports, or use custom network configurations to be able to get data to their application.

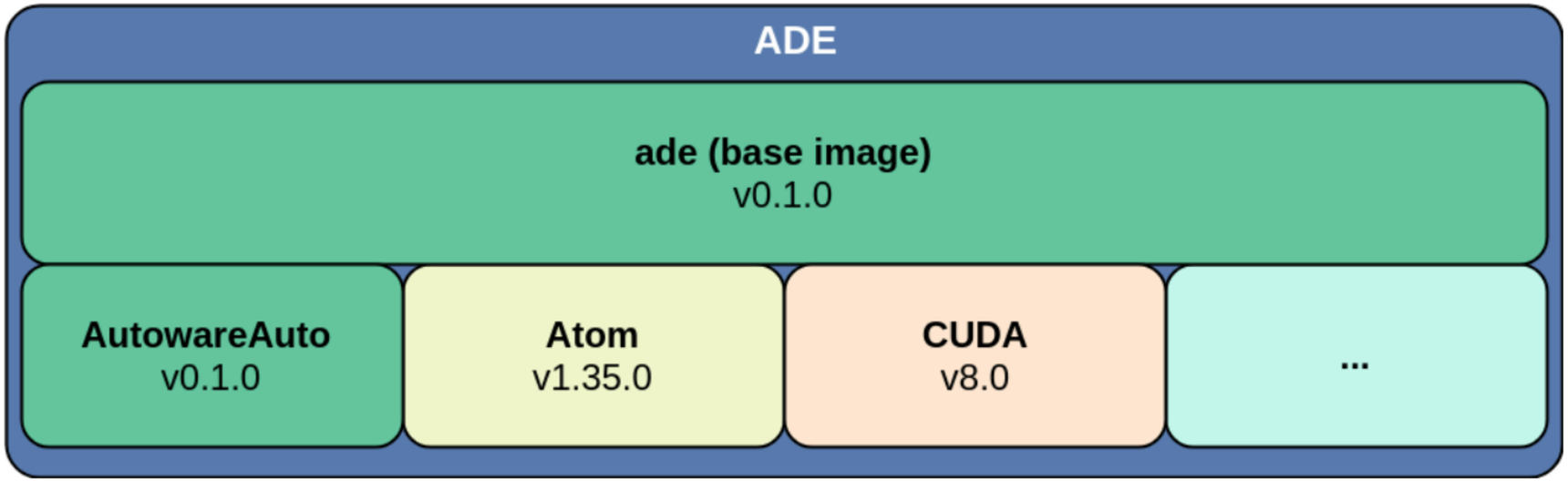
Instead of writing complicated (and limited) wrappers around native Docker commands, **ade-cli** allows using native Docker commands to make these devices available inside the development environment

Terminology

- Docker allows the creation of containers with isolated resources from the host OS
- Images are snapshots of the filesystem for a container. They are based on a Dockerfile, and are created using docker build. Images can be uploaded to a *registry*, where they can be downloaded by others.
- A container is a running instance of an image. When a container is deleted, any changes to its filesystem are usually discarded. A container can publish parts of its filesystem as *volumes*, which can then be mounted by other containers.
- A registry is a server-side application that stores pre-built Docker images
- FQN is the term for the full URL-like path that describes an image. e.g. registry.gitlab.com/apexai/ade-atom:latest
- A tag is a specific version of an image. e.g. latest in the FQN above

Terminology

- ADE creates a Docker container from a base image and mounts additional read-only volumes in /opt. The base image provides development tools (e.g. vim, udpreplay, etc.), and the volumes provide additional development tools (e.g., IDEs, large third-party libraries) or released software versions



Terminology

- The first row of the table is the *base image*, all other entries are *volumes* and have a corresponding entry in /opt. Each row displays the image name, version information (git hash or tag), the Docker tag, and the FQN for each image. If no version information was provided during the creation of the Docker image, the second column will be empty.

```
ade                | v0.1.0   | v0-1-0  | registry.gitlab.com/autoware
autowareauto       | v0.1.0   | v0-1-0  | registry.gitlab.com/autoware
atom               | v1.35.0  | latest  | registry.gitlab.com/apexai/a
cuda               | v8.0     | v8-0    | registry.private-gitlab/cuda
```

Installare ADE

- Docker
- To make ade available globally, install it somewhere in your PATH: * echo $PATH will print a list of possible paths * Commonly used paths are ~/.local/bin and /usr/local/bin
- Download the statically-linked binary from the Releases page of the ade-cli project: e.g. for x86_64
  - $ cd /path/from/step/above
  - $ wget https://gitlab.com/ApexAI/ade-cli/-/jobs/1341322851/artifacts/raw/dist/ade+x86_64
  - $ mv ade+x86_64 ade
  - $ chmod +x ade
  - $ ./ade --version
    4.3.0
  - $ ./ade update-cli
  - $ ./ade --version
    <latest-version>

Installare ADE

- To enable autocompletion, add the following to your .zshrc or .bashrc:

```
if [ -n "$ZSH_VERSION" ]; then
    eval "$(_ADE_COMPLETE=source_zsh ade)"
else
    eval "$(_ADE_COMPLETE=source ade)"
fi
```

ADE e GITlab

The ADE Development Environment (ADE) uses docker and gitlab to manage environments of per project development tools and optional volume images. Volume images may contain additional development tools or released software versions. It enables easy switching of branches for all images

ADE Home

1. ADE needs a directory on the host which will be mounted as the user's home directory within the container. It will be populated with dotfiles and must be different than the user's home directory on the host. In case you use ADE for multiple projects it is recommended to use dedicated ADE home directories for each project.
2. ADE will look for a directory containing a file named .adehome starting with the current working directory and continuing with the parent directories to identify the ADE home directory to be mounted.

```
mkdir adehome
cd adehome
touch .adehome
```

ADE Home

1. ADE needs a directory on the host which will be mounted as the user's home directory within the container. It will be populated with dotfiles and must be different than the user's home directory on the host. In case you use ADE for multiple projects it is recommended to use dedicated ADE home directories for each project.
2. ADE will look for a directory containing a file named .adehome starting with the current working directory and continuing with the parent directories to identify the ADE home directory to be mounted.

ADE files

- To setup ADE for a project, prepare two main components:
  - A **base-image** which is the Docker image that provides all the programs and dependencies for the project. ——> **Dockerfile**
  - An .aderc file which specifies the Docker images to start and any additional docker run arguments.
  - In addition to a **base-image** and an .aderc file, it is recommended to use **volumes** to provide large, stand-alone programs and libraries. **Volumes** allow the project to update different components of ADE without requiring users to download an increasingly large image

ADE files

- Dockerfile: an existing docker image
- env.sh - Configures the default environment on ade enter
- Entrypoint file

env.sh

```
#!/usr/bin/env bash
#
# Copyright 2017 - 2018 Ternaris
# SPDX-License-Identifier: Apache 2.0

for x in /opt/*; do
    if [[ -e "$x/.env.sh" ]]; then
        source "$x/.env.sh"
    fi
done

cd
```

```
#!/usr/bin/env bash

source "/opt/ros/$ROS_DISTRO/setup.bash"

for x in /opt/*; do
    if [[ -e "$x/.env.sh" ]]; then
        source "$x/.env.sh"
    fi
done

cd

# Make colcon mixins available
if [ ! -e ~/.colcon/mixin_repositories.yaml ]; then
    echo "Preparing colcon..."
    colcon mixin add default https://raw.githubusercontent.com/colcon/colcon-mixin-repository/master/index.yaml
    colcon mixin update default > /dev/null
    echo "Done"
fi

# Enable colcon autocompetion
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash

# Add color and git branch to prompt
source ~/.git-prompt.sh
export PROMPT_COMMAND='__git_ps1 "\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[

alias build='colcon build --symlink-install '
alias select='--packages-select '
alias upto='--packages-up-to '
```

entrypoint

```
#!/usr/bin/env bash
#
# Copyright 2017 - 2018 Ternaris
# SPDX-License-Identifier: Apache 2.0

set -e

if [[ -n "$GITLAB_CI" ]]; then
    exec "$@"
fi


if [[ -n "$DEBUG" ]]; then
    set -x
fi


if [[ -n "$TIMEZONE" ]]; then
    echo "$TIMEZONE" > /etc/timezone
    ln -sf /usr/share/zoneinfo/"$TIMEZONE" /etc/localtime
    dpkg-reconfigure -f noninteractive tzdata
fi


groupdel "$GROUP" &>/dev/null || true
groupadd -og "$GROUP_ID" "$GROUP"

useradd -M -u "$USER_ID" -g "$GROUP_ID" -d "/home/$USER" -s /bin/bash "$USER"

groupdel video &> /dev/null || true
groupadd -og "${VIDEO_GROUP_ID}" video
gpasswd -a "${USER}" video
```

https://gitlab.com/ApexAI/minimal-ade/-/blob/master/entrypoint?ref_type=heads

entrypoint

1. There are a few lines to help debug the `entrypoint`:

```
set -e
```

```
if [[ -n "$DEBUG" ]]; then
    set -x
fi
```

2. The Docker image should behave like a regular Docker image, if it's running in CI:

```
if [[ -n "$GITLAB_CI" ]]; then
    exec "$@"
fi
```

   This ensures that ADE and CI can work together to provide developers a single, reproducible environment.

entrypoint

3. The user's environment is configured inside ADE:

   Set the timezone

```
if [[ -n "$TIMEZONE" ]]; then
    echo "$TIMEZONE" > /etc/timezone
    ln -sf /usr/share/zoneinfo/"$TIMEZONE" /etc/localtime
    dpkg-reconfigure -f noninteractive tzdata
fi
```

   This ensures the programs, like `git commit` will have the correct time.

entrypoint

Create a user inside the container and make sure it's in the correct groups

```
groupdel "$GROUP" &>/dev/null || true
groupadd -og "$GROUP_ID" "$GROUP"

useradd -M -u "$USER_ID" -g "$GROUP_ID" -d "/home/$USER" -s /b

groupdel video &> /dev/null || true
groupadd -og "${VIDEO_GROUP_ID}" video
gpasswd -a "${USER}" video


for x in /etc/skel/.*; do
    target="/home/$USER/$(basename "$x")"
    if [[ ! -e "$target" ]]; then
        cp -a "$x" "$target"
        chown -R "$USER":"$GROUP" "$target"
    fi
done
```

This ensures that the user on the host and inside ADE behave the same way. This step includes creating a home directory based on `/etc/skel/`, not unlike a Linux distribution creates a home directory

entrypoint

Create a user inside the container and make sure it's in the correct groups

```
groupdel "$GROUP" &>/dev/null || true
groupadd -og "$GROUP_ID" "$GROUP"

useradd -M -u "$USER_ID" -g "$GROUP_ID" -d "/home/$USER" -s /bin/bash "$USER"

groupdel video &> /dev/null || true
groupadd -og "${VIDEO_GROUP_ID}" video
gpasswd -a "${USER}" video


for x in /etc/skel/.*; do
    target="/home/$USER/$(basename "$x")"
    if [[ ! -e "$target" ]]; then
        cp -a "$x" "$target"
        chown -R "$USER":"$GROUP" "$target"
    fi
done
```

This ensures that the user on the host and inside ADE behave the same way. This step includes creating a home directory based on /etc/skel/, not unlike a Linux distribution creates a home directory

entrypoint

4. It calls the `.adeinit` scripts of any volumes:

```
if [[ -z "$SKIP_ADEINIT" ]]; then
    for x in /opt/*; do
        if [[ -x "$x/.adeinit" ]]; then
            echo "Initializing $x"
            sudo -Hu "$USER" -- bash -lc "$x/.adeinit"
            echo "Initializing $x done"
        fi
    done
fi
```

This gives a way for volumes to provide scripts that must be run on startup to configure the ADE instance appropriately for the volume. See Creating a custom volume also.

entrypoint

5. Finally, it starts ADE:

```
echo 'ADE startup completed.'
exec "$@"
```

entrypoint

Once the `Dockerfile` is ready, build the image by running `docker build` in the directory that contains the `Dockerfile`:

```
docker build -t <image_name>:<tag> .
```

Add `--label ade_image_commit_sha=<git-hash>` and/or `--label ade_image_commit_tag=<git-tag>` to embed VCS information in the Docker image:

```
docker build \
  --label ade_image_commit_sha=<git-hash> \
  --label ade_image_commit_tag=<git-tag> \
  -t <image_name>:<tag> .
```

Gitlab

Name	Last commit
.gitlab-ci.yml	Use newer version of dind
Dockerfile	Upgrade to ROS2 Iron
README.md	Add start of README
apt-packages	Update dependency list
colcon-defaults.yaml	Use colcon defaults and mixins for default arg...
entrypoint	Start of Bold Hearts ADE base image
env.sh	ROS2 alias commands
git-prompt.sh	Add color and git branch to prompt

.aderc

- it is possible to configure ADE using environment variables. In order to make these configurations project-wide, add the environment variables to the .aderc. At a minimum, the .aderc file should include the list of images

```
export ADE_IMAGES="
  image:latest
"
```

Often, it will also need to include the Gitlab instance and registry, so `ade` knows where to download images:

```
export ADE_GITLAB=gitlab.com
export ADE_REGISRTY=registry.gitlab.com
export ADE_IMAGES="
  registry.gitlab.com/autowareauto/autowareauto/ade:v0-1-0
  registry.gitlab.com/autowareauto/autowareauto:v0-1-0
  registry.gitlab.com/apexai/ade-atom:latest
"
```

.aderc

.aderc 251 B

```
# Uncomment the following line to enable access to your camera
# export ADE_DOCKER_RUN_ARGS="--device /dev/video0"
export ADE_GITLAB=gitlab.com
export ADE_REGISTRY=registry.gitlab.com
export ADE_IMAGES="
  registry.gitlab.com/boldhearts/bh-ade:foxy
"

```

Esempio

- https://gitlab.com/ApexAI/minimal-ade

THANK YOU FOR YOUR ATTENTION